# Deep Inverse Models for Non-Intrusive Load Monitoring: Research Position, Problem Formulation, and Recommended Direction

November 3, 2026

**Abstract**

Non-intrusive load monitoring (NILM) estimates individual appliance power trajectories from a whole-house aggregate measurement. This report examines deep forward modelling and inverse optimisation for five target appliances in a house containing other, unrecorded devices. The aim is to recover the correct power values, event shapes, and operating states while explaining the main-meter reading. A simple forward sum does not rule out inverse modelling: it means that the useful learned component should describe appliance behaviour rather than relearn addition. Learned appliance priors, power balance, and background modelling can then guide inverse inference. The report explains Neural Adjoint, compares related methods, and gives numerical examples of gradients, ambiguous decompositions, false pulses, and state consistency. It proposes a staged comparison with direct supervised NILM. Better performance and research novelty remain questions to test.

## 1 Purpose and scope

This report consolidates the project discussion around *deep forward modelling with inverse optimisation*. The intended application is non-intrusive load monitoring (NILM) with five labelled target appliances—kettle, refrigerator, dishwasher, washing machine, and microwave—while other household appliances remain unlabelled and contribute to the whole-house measurement. The project aims to estimate correct device usage with reasonable waveforms, consistent operating states, and an explanation of the measured aggregate. The proposed formulation has not yet been experimentally validated.

The report addresses four questions:

(1) What do forward modelling and inverse optimisation mean?

(2) Why is a learned "appliance powers → aggregate power" network insufficient by itself?

(3) When are deep inverse models normally useful, and are those conditions present in NILM?

(4) What is a defensible research formulation if the aim is to use learned, physically plausible appliance shapes?

## 2 Forward and inverse problems

Let $\mathbf{x}$ be an unobserved input and $\mathbf{y}$ an observed output. A forward process is

$$\mathbf{y} = f(\mathbf{x}). \tag{1}$$

An inverse problem seeks one or more inputs that could have generated an observed output $\mathbf{y}_{\text{obs}}$. A common formulation is

$$\hat{\mathbf{x}} \in \arg\min_{\mathbf{x}} \mathcal{L}\big(f(\mathbf{x}), \mathbf{y}_{\text{obs}}\big) + R(\mathbf{x}), \tag{2}$$

where $\mathcal{L}$ measures mismatch to the observation and $R$ encodes prior knowledge or constraints.

In the terminology used here, the appliance trajectories are the hidden input and the main-meter reading is the observation. For a window of $T$ samples, define

$$\mathbf{p}_i = [p_i(1), \ldots, p_i(T)]^\top \in \mathbb{R}^T, \qquad i = 1, \ldots, 5, \tag{3}$$

$$\mathbf{y}_{\text{agg}} = [y_{\text{agg}}(1), \ldots, y_{\text{agg}}(T)]^\top \in \mathbb{R}^T. \tag{4}$$

Here $p_i(t)$ and $y_{\text{agg}}(t)$ are active powers in watts. The physical active-power balance is

$$y_{\text{agg}}(t) = \sum_{i=1}^{5} p_i(t) + r(t), \tag{5}$$

where $r(t)$ is the effective residual: power from non-target appliances plus measurement, alignment, and modelling discrepancies. Channels must be time-aligned and use compatible active-power measurements. Physical consumption is nonnegative in a house without export, but the effective residual can be negative because of discrepancies. Generation or storage, if present, requires explicit accounting. In vector form,

$$\mathbf{y}_{\text{agg}} = A\mathbf{x} + \mathbf{r}, \qquad \mathbf{x} = [\mathbf{p}_1^\top, \ldots, \mathbf{p}_5^\top]^\top \in \mathbb{R}^{5T}, \tag{6}$$

where $A = [I_T\ I_T\ I_T\ I_T\ I_T]$ and $I_T$ is the $T \times T$ identity matrix.

# 3 What Neural Adjoint (NA) does

Ren et al. [1] named and benchmarked NA, building on earlier uses of learned forward surrogates for inverse design. This section explains the procedure step by step because it is easy to confuse it with ordinary neural-network training. The paper did not introduce the general idea of an inverse problem.

The notation used by Ren et al. is

$$\mathbf{y} = f(\mathbf{x}). \tag{7}$$

Here $\mathbf{x}$ is a controllable but unknown input, $f$ is the real forward process, and $\mathbf{y}$ is the observable output. In their metamaterial example, $\mathbf{x}$ contains geometric design parameters, $f$ is an electromagnetic simulator, and $\mathbf{y}$ is a reflection spectrum. Their goal is: given a desired spectrum, find geometry values that produce it.

## 3.1 Stage A: learn a differentiable replacement for the forward process

Assume that a set of forward input–output pairs is available:

$$\mathcal{D}_{\text{forward}} = \Big\{\big(\mathbf{x}^{(m)}, \mathbf{y}^{(m)}\big)\Big\}_{m=1}^{M}, \qquad \mathbf{y}^{(m)} = f\big(\mathbf{x}^{(m)}\big). \tag{8}$$

The first task is ordinary supervised regression. Train a neural network $\hat{f}_\theta$ so that it imitates the forward process:

$$\theta^* \in \arg\min_{\theta} \frac{1}{M} \sum_{m=1}^{M} \Big\| \hat{f}_\theta\big(\mathbf{x}^{(m)}\big) - \mathbf{y}^{(m)} \Big\|_2^2. \tag{9}$$

Read Equation (9) as follows:

1. Take one known input $\mathbf{x}^{(m)}$ from the forward training set.
2. Put it into the network and obtain a predicted output $\hat{f}_\theta(\mathbf{x}^{(m)})$.
3. Compare that prediction with the known true output $\mathbf{y}^{(m)}$.
4. Change the *network weights* $\theta$ to reduce the squared difference.
5. Repeat this across many examples until the learned network $\hat{f}_{\theta^*}$ is an accurate enough forward surrogate in the relevant input region.

At this stage, the network answers a forward question only:

"If this candidate input were real, what output would it produce?"

It does *not* yet solve the inverse problem. For example, a metasurface surrogate receives eight geometry variables and predicts a 300-dimensional spectrum. The reason to learn $\hat{f}_\theta$ is that the real electromagnetic simulator is expensive and does not conveniently provide the gradient needed for design optimisation.

### 3.2 Stage B: freeze the network and optimise its input

Now suppose a new desired or observed output $\mathbf{y}_{\text{obs}}$ is given. This is the inverse-problem stage. The trained weights are frozen:

$$\theta = \theta^*. \tag{10}$$

"Frozen" means that no gradient step is allowed to change a convolution kernel, a linear-layer weight, or any other network parameter. Instead, initialise a candidate input

$$\hat{\mathbf{x}}^{(0)} \sim \Gamma, \tag{11}$$

where $\Gamma$ is a chosen initialisation distribution. The initial candidate can be random, can come from a direct inverse model, or can be a physically motivated guess.

For the current candidate $\hat{\mathbf{x}}^{(k)}$, perform the following six operations:

1. **Forward pass.** Compute the predicted output

$$\hat{\mathbf{y}}^{(k)} = \hat{f}_{\theta^*}\big(\hat{\mathbf{x}}^{(k)}\big). \tag{12}$$

2. **Measure mismatch.** For example, use squared error:

$$\ell^{(k)} = \left\| \hat{\mathbf{y}}^{(k)} - \mathbf{y}_{\text{obs}} \right\|_2^2. \tag{13}$$

3. **Differentiate with respect to the input.** Automatic differentiation computes $\nabla_{\mathbf{x}} \ell^{(k)}$. This is a vector with one entry per candidate-input variable.
4. **Interpret the sign.** A positive gradient entry means that increasing that input slightly increases the loss locally, so the update tends to decrease it. A negative gradient entry means that increasing the input can reduce the loss locally, so the update tends to increase it.
5. **Update the input.** Apply the gradient-descent rule below.
6. **Repeat.** Stop when the loss no longer improves sufficiently, a maximum iteration count is reached, or another stopping criterion is met. Multiple initialisations can be optimised independently because the problem can have local minima.

The update rule is

$$\hat{\mathbf{x}}^{(k+1)} = \hat{\mathbf{x}}^{(k)} - \alpha_k \, \nabla_{\mathbf{x}} \mathcal{L}\big(\hat{f}_{\theta^*}(\mathbf{x}), \mathbf{y}_{\text{obs}}\big)\Big|_{\mathbf{x}=\hat{\mathbf{x}}^{(k)}} . \tag{14}$$

This report's Equation (14) is the same input-gradient mechanism as Equation (4) in Ren et al. [1], written with explicit vector-gradient notation. It comes from applying ordinary gradient descent to the composite loss

$$\ell(\mathbf{x}) = \mathcal{L}\big(\hat{f}_{\theta^*}(\mathbf{x}), \mathbf{y}_{\text{obs}}\big). \tag{15}$$

The chain rule carries the error gradient from the output, through the frozen neural network, to its input. The index $k$ is the optimisation iteration, not physical time; $\alpha_k > 0$ is the step size and can be selected or adapted by an optimiser such as Adam.

**A one-variable numerical example.** Assume temporarily that the learned forward model is $\hat{f}(x) = 2x$, the desired output is $y_{\text{obs}} = 10$, and the loss is

$$\ell(x) = (2x - 10)^2. \tag{16}$$

Start with $x^{(0)} = 3$. The forward prediction is $\hat{f}(3) = 6$, so the loss is $(6 - 10)^2 = 16$. The input gradient is

$$\frac{d\ell}{dx} = 4(2x - 10). \tag{17}$$

At $x = 3$, this gradient is $-16$. With $\alpha = 0.1$, the update is

$$x^{(1)} = 3 - 0.1(-16) = 4.6. \tag{18}$$

The new prediction is $2(4.6) = 9.2$, closer to 10. The important point is that $x$ changed; the parameters of $\hat{f}(x) = 2x$ did not change. NA applies this same idea to many input variables and a complex learned forward network.

### 3.3 Boundary loss and its purpose

The authors also introduced a boundary loss in their Equation (5). A componentwise version is

$$\mathcal{L}_{\text{bnd}} = \text{ReLU}\left(|\hat{\mathbf{x}} - \boldsymbol{\mu}_x| - \frac{1}{2}\mathbf{R}_x\right), \tag{19}$$

where $\boldsymbol{\mu}_x$ and $\mathbf{R}_x$ are the training-input mean and range. The absolute-value term measures how far a candidate has moved from the centre of the training data. Subtracting half the range creates a zero-penalty region approximately corresponding to the training interval; ReLU then keeps only positive violations.

For example, suppose one design variable was sampled uniformly between 0 and 1800. Its mean is 900 and its range is 1800. A candidate value of 1500 gives

$$\text{ReLU}(|1500 - 900| - 900) = \text{ReLU}(-300) = 0, \tag{20}$$

so it is not penalised. A value of 2500 gives

$$\text{ReLU}(|2500 - 900| - 900) = \text{ReLU}(700) = 700, \tag{21}$$

so it receives a positive penalty. The purpose is not to prove that every in-range solution is correct; it is to prevent the optimiser from exploiting a neural surrogate in an untrained region where its prediction may be false.

### 3.4 When this is useful, and what changes for NILM

The method is useful when the forward process is expensive or lacks readily available analytic gradients, yet it can be accurately approximated by a differentiable surrogate. Ren et al. [1] studied inverse design tasks such as metamaterial design, where geometric variables map nonlinearly to high-dimensional electromagnetic spectra. The desired outcome is often *any* feasible design producing the target response, not recovery of a unique historical cause.

For low-frequency power-only NILM, the simple physical forward relation is already addition. Therefore, training a neural network merely to reproduce appliance-power summation does not create the informative complex forward model present in Ren et al.'s problem. The NA update mechanism is still mathematically valid, but it does not solve appliance identity by itself: additional learned appliance-shape priors, background modelling, or richer measurements are required. The remaining sections formulate that distinction.

## 4 Why learning the power sum is not a sufficient NILM method

For low-frequency active-power NILM, the essential forward relation in Equation (5) is already known. If a neural network is trained with appliance powers and residual as inputs and aggregate power as the target, its easiest and correct solution is simply

$$F_\theta(\mathbf{p}_1, \ldots, \mathbf{p}_5, \mathbf{r}) \approx \sum_{i=1}^{5} \mathbf{p}_i + \mathbf{r}. \tag{22}$$

For this formulation, exact addition is a clear baseline. A learned replacement would need to show a useful benefit. A simple addition operator does not make inverse modelling unsuitable: the useful information can come from learned models of the individual sources, as in source separation with deep generative priors [5].

More importantly, exact reconstruction of aggregate power does not identify appliances. For the sum operator,

$$\frac{\partial y_{\text{agg}}(t)}{\partial p_i(t)} = 1 \qquad \text{for every target appliance } i. \tag{23}$$

Thus, aggregate mismatch tells an optimiser that the total needs to change, but it does not specify whether the kettle, refrigerator, microwave, or residual should change. Any perturbation satisfying

$$A\Delta\mathbf{x} + \Delta\mathbf{r} = 0 \tag{24}$$

leaves the aggregate unchanged.

For example, an observed 2000 W sample may have the following explanations:

$$\begin{aligned}
2000 &= 100 \text{ W refrigerator} + 1500 \text{ W microwave} + 400 \text{ W residual},\\
2000 &= 100 \text{ W refrigerator} + 1900 \text{ W microwave} + 0 \text{ W residual},\\
2000 &= 0 \text{ W targets} + 2000 \text{ W residual}.
\end{aligned}$$

Every line has zero aggregate reconstruction error. Only the first may be the true appliance allocation. This is an *identifiability* problem, not a failure of gradient descent.

Adding an unrestricted residual variable completes the physical accounting but creates a second degeneracy: the optimiser can set all target appliances to zero and assign all measured power to the residual. Conversely, heavily penalising residual magnitude can incorrectly assign genuine non-target consumption to the five targets. A residual model must therefore be constrained and empirically tested; it cannot be assumed to be always small or smooth.

# 5 Direct NILM and inverse optimisation are different tools

A direct supervised NILM model learns

$$[\hat{\mathbf{p}}_1, \ldots, \hat{\mathbf{p}}_5] = D_\phi(\mathbf{y}_{\text{agg}}). \tag{25}$$

A convolutional neural network (CNN) can learn temporal features such as step changes and short plateaus. Other temporal architectures can also use context across a window. Kelly and Knottenbelt [10] used convolutional layers in direct NILM and also studied predicting the start, end, and average power of appliance activations. With enough labelled training pairs, a direct model is a strong accuracy and speed baseline. A CNN can learn typical patterns, but its architecture alone does not guarantee correct event duration or state consistency.

**Does disaggregation violate physical logic?** No. Appliances produce the total electrical load; this is the forward direction of the measurement process. Estimating appliances from the total is inference about hidden causes. A direct model does not claim that the aggregate physically causes an appliance to turn on. Both direct prediction and optimisation can solve the same inverse task. The useful research question is whether explicit forward consistency and device rules improve that inference. Existing NILM work already combines additive observations, device states, and inverse inference [6, 7].

Deep inverse optimisation is not designed to replace Equation (25) in every setting. It is normally attractive when at least one of the following conditions holds:

- a reliable, informative forward model or simulator exists;
- inverse labels are scarce, unavailable, or expensive;
- multiple inputs can explain the same observation and several feasible solutions or uncertainty estimates are needed;
- hard physical, safety, manufacturing, or consistency constraints must be imposed at inference;
- iterative test-time computation is acceptable.

Ren et al. [1] illustrate inverse design and robotic control tasks; Adler and Öktem [8] study learned reconstruction using an imaging operator. These applications show different reasons to use forward knowledge. In NILM, addition is simple but appliance behaviour is structured. This makes learned behaviour priors a reasonable research direction, while leaving open whether iterative inference improves on a direct model. Multi-start optimisation can find several solutions, but calibrated uncertainty requires a probabilistic formulation; it does not follow from using several starts alone.

## 5.1 Four related approaches and what they change

These are related choices, not four names for the same method. None guarantees correct appliance recovery.

| Approach | Main idea | Role in this project |
|---|---|---|
| NA [1] | Freeze a learned forward model and optimise its unknown inputs at test time. | Closest to the supervisor's input-optimisation description. |
| Tandem network [2] | Train an inverse network through a frozen forward model. | Fast direct prediction with forward reconstruction during training. Appliance labels are still useful. |
| Learned iterative reconstruction [8] | Learn parts of a finite reconstruction procedure that uses a forward operator. | A possible way to learn a small number of correction steps. |
| Invertible network [9] | Augment observations with latent variables and learn an invertible relation. | A way to represent several possible inputs; sampling replaces ordinary input optimisation. |

The following formulation develops one possible route: learned appliance generators followed by inverse optimisation. This is a proposed NILM adaptation, not a result established by the papers above.

Ren et al. [1], Section 4, compare a mixture density network (MDN), a conditional variational auto-encoder (cVAE), an invertible neural network (INN), a conditional invertible neural network (cINN), and a tandem model. Here $\mathbf{x}$ is the hidden input, $\mathbf{y}$ is the observation, and $\mathbf{z}$ supplies latent variation. The symbol $D_{\mathrm{KL}}$ denotes Kullback–Leibler (KL) divergence.

## 5.2 Comparison of the five benchmark inverse models

Table 1: Learning objectives and inference procedures for the benchmark models in Ren et al. [1], Section 4.

| Model | What is learned? | Training loss and its purpose | What happens at test time? | Several answers? |
|---|---|---|---|---|
| **MDN** | A distribution of hidden inputs: Gaussian means, covariances, and mixture weights depend on $\mathbf{y}$. | $-\log\left[\sum_{j=1}^{K}\pi_j(\mathbf{y})\,\mathcal{N}\big(\mathbf{x};\boldsymbol{\mu}_j(\mathbf{y}),\Sigma_j(\mathbf{y})\big)\right]$<br>Make the labelled $\mathbf{x}$ likely under the predicted mixture. | Predict the mixture from $\mathbf{y}$; sample a component and then sample $\hat{\mathbf{x}}$ from that Gaussian. | **Yes.**<br>New draws give candidates. $K$ is chosen in advance. |
| **cVAE** | An encoder $q_\phi(\mathbf{z}\mid\mathbf{x},\mathbf{y})$ and a decoder $g_\psi(\mathbf{y},\mathbf{z})$. | $\mathbb{E}_{q_\phi}\,\lVert g_\psi(\mathbf{y},\mathbf{z})-\mathbf{x}\rVert_2^2$<br>$+\,\beta D_{\mathrm{KL}}(q_\phi\Vert p_0)$<br>Reconstruct $\mathbf{x}$ while regularising the latent distribution. | Draw $\mathbf{z}\sim p_0$ and decode $\hat{\mathbf{x}}=g_\psi(\mathbf{y},\mathbf{z})$. The encoder is used during training. | **Yes.**<br>Different latent draws can produce different answers. |
| **INN** | A bijection $h_\theta(\mathbf{x})=(\hat{\mathbf{y}},\mathbf{z})$ with matched total dimensions. | $\dfrac{\lVert\hat{\mathbf{y}}-\mathbf{y}\rVert_2^2}{2\sigma_y^2}+\dfrac{1}{2}\lVert\mathbf{z}\rVert_2^2$<br>$-\log\lvert\det J_h\rvert$<br>Match the response, fit a Gaussian latent, and account for volume change. | Fix $\mathbf{y}$, draw $\mathbf{z}\sim\mathcal{N}(0,I)$, and evaluate $h_\theta^{-1}(\mathbf{y},\mathbf{z})$. | **Yes.**<br>Extra latent variables represent lost information. |
| **cINN** | For fixed $\mathbf{y}$, a bijection $h_\theta(\mathbf{x};\mathbf{y})=\mathbf{z}$. | $\dfrac{1}{2}\lVert\mathbf{z}\rVert_2^2-\log\lvert\det J_h\rvert$<br>Fit a conditional density. Both $\mathbf{z}$ and $J_h$ depend on $\mathbf{y}$. | Fix $\mathbf{y}$ as a condition, draw Gaussian $\mathbf{z}$, and evaluate $h_\theta^{-1}(\mathbf{z};\mathbf{y})$. | **Yes.**<br>The observation is a condition, not an extra output. |
| **Tandem** | First train $\hat{f}$; then freeze it and train the inverse predictor $g_\psi$. | $\left\lVert\hat{f}(g_\psi(\mathbf{y}))-\mathbf{y}\right\rVert_2^2$<br>$+\,\mathcal{L}_{\mathrm{bnd}}(g_\psi(\mathbf{y}))$<br>The predicted input should reproduce the required response within the allowed range. | One inverse-network pass gives $\hat{\mathbf{x}}=g_\psi(\mathbf{y})$. No input-optimisation loop is required. | **One.**<br>The deterministic model in this benchmark returns the same answer for the same $\mathbf{y}$. |

**Where NA differs:** NA freezes a trained forward network and iteratively changes the candidate input $\mathbf{x}$. The five models above normally generate candidates by sampling or direct network evaluation, with weights fixed at test time. Multiple candidates do not by themselves establish calibrated uncertainty or correct NILM attribution.

### 5.3 How to read the comparison table

Table 1 describes the benchmark methods, not validated NILM implementations. The first four model distributions of possible inputs. The tandem model seeks one input that reproduces the observation. Their objectives use labelled input–output pairs during training; none automatically solves the unknown-background problem in this project. Losses are written per training pair with fixed additive constants omitted. The cVAE row uses standard Gaussian variational notation, explained below, rather than a literal transcription of the printed formula.

**MDN: choosing among several possible explanations.** The component count is $K$. Each mixing weight satisfies $\pi_j \geq 0$ and $\sum_j \pi_j = 1$; $\boldsymbol{\mu}_j$ and $\Sigma_j$ are the component mean and covariance. The table uses the standard Gaussian density, including its normalisation, to express the negative log likelihood compactly. Ren et al.'s Equation (6) writes this in expanded form up to fixed constants. For illustration, a two-component model might assign probability mass near two different appliance allocations that both explain a 2000 W aggregate. Sampling can reveal both explanations, but a Gaussian component does not automatically encode realistic appliance event shapes.

**cVAE: training reconstruction and test-time sampling are different.** The encoder sees both $\mathbf{x}$ and $\mathbf{y}$ during training. At test time $\mathbf{x}$ is unknown, so the decoder uses $\mathbf{y}$ and a draw from the chosen prior $p_0$, here $\mathcal{N}(0, I)$. The reconstruction-plus-KL objective comes from the negative evidence lower bound (ELBO). The table uses squared reconstruction error for a fixed-variance Gaussian decoder and a tunable weight $\beta$. The standard unweighted ELBO corresponds to the appropriate likelihood scaling and $\beta = 1$.

For a diagonal Gaussian encoder with variances $v_j > 0$, the standard KL term is [13]

$$D_{\mathrm{KL}}(q_\phi \| \mathcal{N}(0, I)) = \frac{1}{2} \sum_j \left( \mu_j^2 + v_j - \log v_j - 1 \right). \tag{26}$$

**Meaning and example.** This penalty is zero when the encoder has $\mu_j = 0$ and $v_j = 1$ in every coordinate. For one coordinate with $\mu = 1$ and $v = 1$, it is 0.5. The table deliberately uses this standard notation rather than copying the ambiguous variance notation and signs printed in Ren et al.'s Equation (7). It is an explanatory standard formulation, not a verbatim transcription of that equation.

**INN and cINN: what the determinant means.** The Jacobian $J_h$ contains derivatives of the learned transformation with respect to $\mathbf{x}$. Its determinant measures local volume change; the log-determinant term comes from the probability change-of-variables rule. The table gives the maximum-likelihood objectives used in the benchmark rather than the alternative distribution-matching training scheme. In the INN row, $\sigma_y > 0$ is a fixed response-error scale. Matching dimensions may require padding or augmentation. For a fixed condition $\mathbf{y}$, the cINN maps between equally sized $\mathbf{x}$ and $\mathbf{z}$ spaces. Its loss has no separate response-error term, but it still uses $\mathbf{y}$ through the conditional network and Jacobian. Mathematical invertibility of the learned network does not undo information loss in the physical aggregate.

**Tandem versus NA: what is updated?** In tandem training, $\hat{f}$ is frozen and the inverse-network weights $\psi$ change. In NA inference, the forward-network weights are frozen and the candidate input changes. In this report $g_\psi$ names a learned inverse predictor; the notation does not assert that the physical forward map has a unique inverse. For NILM, tandem reconstruction alone can still accept a wrong appliance allocation with the correct total. Appliance labels and behaviour constraints remain relevant.

# 6 A more coherent inverse-optimisation route: generative appliance priors

The meaningful deep component can be placed in a model of plausible appliance trajectories rather than in the known addition operator. Let $G_i$ be a learned differentiable generator or decoder for appliance $i$:

$$\mathbf{p}_i = G_i(\mathbf{z}_i), \qquad \mathbf{z}_i \in \mathbb{R}^{d_i}. \tag{27}$$

The code $\mathbf{z}_i$ is an input that controls a complete generated trajectory. A lower dimension, $d_i < T$, can restrict the search, but strong compression is an optional design choice: it can also remove real variations. Latent coordinates need not have interpretable meanings. A CNN decoder can learn class-specific shapes from labelled appliance windows. The training set must include off periods, partial events, and relevant operating modes rather than only complete on cycles. A background generator $G_{\text{bg}}$ may be trained from residual windows computed on training data only:

$$\mathbf{r}^{(m)} = \mathbf{y}_{\text{agg}}^{(m)} - \sum_{i=1}^{5} \mathbf{p}_i^{(m)}. \tag{28}$$

For a new aggregate, a latent-space inverse formulation is

$$\begin{aligned} \min_{\mathbf{z}_1,\ldots,\mathbf{z}_5,\mathbf{z}_{\text{bg}}} \quad & \left\| \sum_{i=1}^{5} G_i(\mathbf{z}_i) + G_{\text{bg}}(\mathbf{z}_{\text{bg}}) - \mathbf{y}_{\text{agg}} \right\|_2^2 \\ & + \lambda_z \sum_{i=1}^{5} \Omega_i(\mathbf{z}_i) + \lambda_{\text{bg}} \Omega_{\text{bg}}(\mathbf{z}_{\text{bg}}). \end{aligned} \tag{29}$$

Here $\Omega_i$ and $\Omega_{\text{bg}}$ constrain the latent variables or express their learned probability distributions. The exact sum remains the physical forward operator. The generators restrict candidates to their output range; this helps only if they have learned useful device-specific structure. A generator does not guarantee realistic outputs for arbitrary codes. Terms must use declared scaling and validation-selected weights: a watt-squared mismatch and a dimensionless prior cannot be balanced by unspecified numerical weights.

This formulation is related to source separation with generative priors [4, 5] and generative-model-based inverse problems [3]. These papers use different inference methods; a prior-based approach does not automatically mean NA. The proposed formulation uses the NA input-gradient principle when it freezes differentiable generators and optimises their inputs. It is best described as *NILM with generative appliance priors and constrained inverse optimisation.* The generators learn source behaviour; the summation remains exact.

# 7 Detailed worked example: from observation to appliance estimates

This section gives a deliberately small numerical example. It is a teaching example, not a trained model, a benchmark result, or a claim that the listed priors are sufficient in a real house. It illustrates precisely what each formulation knows, what it optimises, and why the CNN or generator must carry useful appliance-specific information.

## 7.1 Step 0: one observed aggregate window

Consider a six-sample window, where each sample represents the average active power over a short interval. The observed aggregate is

$$\mathbf{y}_{\text{agg}} = [400,\ 400,\ 2000,\ 2000,\ 400,\ 400]^\top \text{ W}. \tag{30}$$

For evaluation only, suppose the hidden ground truth is shown in Table 2. The refrigerator is operating throughout the window at 100 W. The microwave is on for samples 3 and 4 at 1500 W. Other target appliances are off. Unrecorded appliances contribute a 300–400 W background. In a real test window, the final six rows of Table 2 are unknown to the algorithm; they are shown only because a supervised NILM dataset permits later evaluation.

Table 2: Illustrative hidden appliance powers. Only the aggregate row is available at test time. All values are watts.

| Component / sample | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| Observed aggregate $\mathbf{y}_{\text{agg}}$ | 400 | 400 | 2000 | 2000 | 400 | 400 |
| Kettle | 0 | 0 | 0 | 0 | 0 | 0 |
| Refrigerator | 100 | 100 | 100 | 100 | 100 | 100 |
| Dishwasher | 0 | 0 | 0 | 0 | 0 | 0 |
| Washing machine | 0 | 0 | 0 | 0 | 0 | 0 |
| Microwave | 0 | 0 | 1500 | 1500 | 0 | 0 |
| Residual $\mathbf{r}$ | 300 | 300 | 400 | 400 | 300 | 300 |

The physical relation is exactly

$$\mathbf{y}_{\text{agg}} = \mathbf{p}_{\text{fridge}} + \mathbf{p}_{\text{microwave}} + \mathbf{r}. \tag{31}$$

For sample 3, Equation (31) becomes $2000 = 100+1500+400$. The other three target trajectories are zero, but this is unknown at inference.

### 7.2 Step 1: what a direct CNN NILM model does

A direct CNN-based NILM model is trained from paired examples,

$$\left(\mathbf{y}_{\text{agg}}^{(m)}, \mathbf{p}_1^{(m)}, \ldots, \mathbf{p}_5^{(m)}\right), \tag{32}$$

to minimise, for example,

$$\mathcal{L}_{\text{direct}}(\phi) = \frac{1}{M}\sum_{m=1}^{M}\sum_{i=1}^{5}\left\| D_{\phi,i}(\mathbf{y}_{\text{agg}}^{(m)}) - \mathbf{p}_i^{(m)} \right\|_2^2. \tag{33}$$

The convolutional filters can learn that a two-sample, approximately 1500 W plateau superimposed on a lower baseline is more consistent with a microwave event than with a refrigerator event. It can also learn that a refrigerator commonly has lower power and recurring on/off behaviour. This is why CNNs are useful: they learn temporal patterns from labelled appliance data.

For the new aggregate in Equation (30), suppose the trained direct model returns the following imperfect initial prediction:

$$\tilde{\mathbf{p}}_{\text{fridge}} = [100, 100, 100, 100, 100, 100]^\top, \tag{34}$$

$$\tilde{\mathbf{p}}_{\text{microwave}} = [0, 0, 1000, 1000, 0, 0]^\top, \tag{35}$$

$$\tilde{\mathbf{p}}_{\text{kettle}} = \tilde{\mathbf{p}}_{\text{dishwasher}} = \tilde{\mathbf{p}}_{\text{washing}} = \mathbf{0}. \tag{36}$$

Its target sum is therefore

$$\sum_{i=1}^{5}\tilde{\mathbf{p}}_i = [100, 100, 1100, 1100, 100, 100]^\top \text{ W}. \tag{37}$$

Relative to the aggregate, the implied unexplained residual is

$$\tilde{\mathbf{r}} = \mathbf{y}_{\text{agg}} - \sum_{i=1}^{5} \tilde{\mathbf{p}}_i = [300, 300, 900, 900, 300, 300]^\top \text{ W}. \tag{38}$$

This is a plausible starting point but not the hidden truth: the microwave is underestimated by 500 W at samples 3 and 4, while the implied residual is too large by the same amount.

### 7.3 Step 2: why summation-only inverse optimisation cannot decide

Suppose we ignore all learned shape information and minimise only power-balance mismatch:

$$\mathcal{J}_{\text{sum}}(\mathbf{x}, \mathbf{r}) = \left\| \sum_{i=1}^{5} \mathbf{p}_i + \mathbf{r} - \mathbf{y}_{\text{agg}} \right\|_2^2. \tag{39}$$

At samples 3 and 4, each of the following candidates has exactly zero loss under Equation (39):

| Candidate | Refrigerator | Microwave | Residual |
|---|---|---|---|
| Hidden truth | 100 W | 1500 W | 400 W |
| Microwave inflation | 100 W | 1900 W | 0 W |
| Background collapse | 0 W | 0 W | 2000 W |
| Target exchange | 100 W | 0 W | 1900 W |

The last row could represent an unrecorded high-power device being assigned entirely to residual while the microwave is incorrectly set to zero. The sum objective cannot prefer one row over another.

The gradient makes the same point. Define the sample-3 reconstruction error

$$e = p_{\text{kettle}} + p_{\text{fridge}} + p_{\text{dishwasher}} + p_{\text{washing}} + p_{\text{microwave}} + r - 2000. \tag{40}$$

For the squared error $e^2$,

$$\frac{\partial e^2}{\partial p_i} = 2e, \qquad \frac{\partial e^2}{\partial r} = 2e. \tag{41}$$

Every component receives the same reconstruction gradient. If the current candidate is the direct-model initialisation in Equation (36) with $r = 200$ W at sample 3, then $e = 100 + 1000 + 200 - 2000 = -700$ W. Gradient descent says to increase every candidate variable; it does not know that the microwave, rather than the refrigerator or residual, should receive the missing 700 W. The physical sum provides conservation, not appliance identity.

### 7.4 Step 3: what a CNN-based generative prior changes

The proposed inverse approach should not optimise every power sample freely. Instead, it freezes appliance-specific decoders after prior training and optimises their latent inputs. For the example, write

$$\mathbf{p}_{\text{fridge}} = G_{\text{fridge}}(\mathbf{z}_{\text{fridge}}), \tag{42}$$

$$\mathbf{p}_{\text{microwave}} = G_{\text{microwave}}(\mathbf{z}_{\text{microwave}}), \tag{43}$$

$$\mathbf{r} = G_{\text{bg}}(\mathbf{z}_{\text{bg}}). \tag{44}$$

Each $G$ can be a CNN decoder, an autoencoder decoder, or another differentiable temporal generator. It is trained before test-time inverse optimisation. During inference, all generator weights are fixed; only $\mathbf{z}_{\text{fridge}}$, $\mathbf{z}_{\text{microwave}}$, and $\mathbf{z}_{\text{bg}}$ change.

For intuition, a well-trained microwave generator may only produce trajectories resembling a finite-duration high-power event, such as

$$G_{\text{microwave}}(\mathbf{z}_{\text{microwave}}) \approx [0, 0, a, a, 0, 0]^\top, \qquad 0 \le a \le 1800 \text{ W}, \tag{45}$$

where the latent code controls timing, duration, and amplitude. A refrigerator generator may favour a low-power trajectory such as

$$G_{\text{fridge}}(\mathbf{z}_{\text{fridge}}) \approx [b, b, b, b, b, b]^\top, \qquad 0 \le b \le 300 \text{ W}. \tag{46}$$

Equations (45)–(46) are simplified illustrations, not recommended hand-crafted models. A real CNN generator must learn a distribution of variable durations, transitions, amplitudes, and operating modes from training data.

The test-time objective becomes

$$\begin{aligned} \mathcal{J}_{\text{prior}} = &\left\| \sum_{i=1}^{5} G_i(\mathbf{z}_i) + G_{\text{bg}}(\mathbf{z}_{\text{bg}}) - \mathbf{y}_{\text{agg}} \right\|_2^2 \\ &+ \lambda_{\text{bg}} \Omega_{\text{bg}}(\mathbf{z}_{\text{bg}}) + \lambda_z \sum_{i=1}^{5} \Omega_i(\mathbf{z}_i). \end{aligned} \tag{47}$$

The first line enforces aggregate consistency. The second line favours latent codes typical of the appliance and background training distributions. It is the latter information that can make a short microwave-shaped event preferable to an implausible 1900 W trajectory or a residual that has to explain the whole window.

Starting from the direct CNN prediction, an optimiser updates latent variables by

$$\mathbf{z}^{(k+1)} = \mathbf{z}^{(k)} - \eta_k \nabla_{\mathbf{z}} \mathcal{J}_{\text{prior}}\big(\mathbf{z}^{(k)}\big), \tag{48}$$

where $\mathbf{z}$ stacks every appliance and background latent variable, and $\eta_k$ is an optimisation step size. This uses the NA principle: the differentiable models are fixed and their inputs are optimised. To initialise from a direct prediction, the predicted trajectory must first be converted to a code. For an autoencoder prior, an encoder $E_i$ can give $\mathbf{z}_i^{(0)} = E_i(\tilde{\mathbf{p}}_i)$. Without an encoder, an initial code can be fitted to the predicted trajectory or sampled. A power trajectory cannot simply be used as a latent code of a different dimension.

### 7.5 Step 4: possible outcomes and interpretation

If the learned priors are accurate and distinguish the signals sufficiently, optimisation may return approximately

$$\hat{\mathbf{p}}_{\text{fridge}} = [100, 100, 100, 100, 100, 100]^\top, \tag{49}$$

$$\hat{\mathbf{p}}_{\text{microwave}} = [0, 0, 1500, 1500, 0, 0]^\top, \tag{50}$$

$$\hat{\mathbf{r}} = [300, 300, 400, 400, 300, 300]^\top. \tag{51}$$

This outcome matches both the aggregate and the hidden target labels. However, it is not guaranteed. If the background generator has learned high-power rectangular events that look similar to a microwave, it may still allocate the event to background. If the microwave generator is too permissive, it may absorb genuine background consumption. These are meaningful failure modes, not implementation bugs.

The correct evaluation is therefore target-wise. For the microwave, for example, the mean absolute error (MAE) over the six samples is

$$\text{MAE}_{\text{microwave}} = \frac{1}{6} \sum_{t=1}^{6} \left| \hat{p}_{\text{microwave}}(t) - p_{\text{microwave}}(t) \right|. \tag{52}$$

The "microwave inflation" candidate has zero aggregate error but a microwave error of 400 W at samples 3 and 4. Hence its microwave MAE is $800/6 \approx 133.3$ W. A low aggregate loss cannot replace this appliance-level evaluation.

### 7.6 What this example does and does not establish

The example establishes the logical roles of the components: exact addition is the power-balance constraint; the direct CNN supplies a fast initial estimate; the generators encode shape priors; and latent optimisation enforces a constrained explanation of the aggregate. It does *not* establish that a specific prior, network architecture, residual model, or optimisation schedule will work on the project data. That is the empirical research question.

## 8 Why waveform shape, operating logic, and forward consistency matter

### 8.1 What the supervisor's direction can mean

The physical story is clear: an appliance runs, produces a power trajectory, and contributes to the main-meter reading. It is reasonable to build a model around this story. For example, a candidate microwave operation might be "start at sample 3, remain active for two intervals, and draw approximately 1500 W." A learned temporal model can turn such a description into a waveform. The forward sum then predicts what the meter would observe.

There is a useful distinction between two tasks:

1. **Given every appliance power sample, calculate the aggregate.** This is addition and needs no neural model.
2. **Given an appliance operation or learned code, generate its power waveform.** This can involve device-specific shapes, modes, timing, and stages, so it can benefit from a deep model.

The second task gives a meaningful learned forward component for the project. The inverse stage searches for operating descriptions or codes whose generated waveforms explain the aggregate. This is a concrete interpretation of the supervisor's suggestion. It changes the learned forward inputs from fully specified appliance powers to a representation of appliance operation.

This direction is plausible. Its justification is that device knowledge can remove some unreasonable explanations of a mixture. A simple mixing operator does not prevent this: source-separation work already combines learned source priors with a known mixture relationship [5]. It remains necessary to measure whether the proposed priors help this five-target setting.

### 8.2 What is physical, and what is learned behaviour?

| Requirement | Meaning in this project | How to use it |
|---|---|---|
| Power balance | Targets plus background explain the meter. | Use compatible channels and a residual that accounts for discrepancies. |
| State–power consistency | A device labelled off should not have a large running-power estimate. | Couple state and power outputs; allow the device's standby consumption. |
| Plausible duration | A very short isolated event may be unlikely for a particular operating mode. | Learn duration statistics or use a validated soft penalty. |
| Plausible shape and stages | A device's event should resemble its own operating behaviour. | Learn temporal priors from varied appliance examples. |

Power balance is a physical accounting relation. Typical duration and stage order are device-behaviour priors. A prediction can satisfy power balance while violating those priors. It can also look plausible while assigning an event to the wrong device. Both agreement with the observation and correct appliance attribution matter. Duration-aware NILM has explicit precedents, including Wong et al. [12].

### 8.3 Example: the same MAE can hide different event errors

For this separate eight-sample example, suppose the true microwave trajectory is

$$\mathbf{p}_{\text{true}} = [0, 0, 0, 1500, 1500, 0, 0, 0]^\top \text{ W}.$$

Compare two predictions:

$$\hat{\mathbf{p}}_A = [0, 0, 0, 1400, 1400, 0, 0, 0]^\top \text{ W},$$
$$\hat{\mathbf{p}}_B = [0, 100, 0, 1500, 1500, 0, 100, 0]^\top \text{ W}.$$

Prediction A finds the correct event but underestimates its amplitude. Prediction B finds that event and adds two false 100 W pulses. Both have the same MAE:

$$\text{MAE}_A = \frac{100 + 100}{8} = 25 \text{ W}, \qquad \text{MAE}_B = \frac{100 + 100}{8} = 25 \text{ W}.$$

MAE does measure pointwise shape and timing errors, but one average does not describe their type. Here the user may care more about the two false activations in B than about the small amplitude bias in A.

Aggregate consistency does not settle the comparison either. Let the refrigerator be 100 W throughout and the aggregate be $[400, 400, 400, 2000, 2000, 400, 400, 400]^\top$ W. Candidate A can fit it exactly with residual $[300, 300, 300, 500, 500, 300, 300, 300]^\top$ W. Candidate B can fit it exactly with residual $[300, 200, 300, 400, 400, 300, 200, 300]^\top$ W. A background model and device-event knowledge are needed to assess these reallocations.

This example does not imply that every small pulse is impossible. Standby activity, partial sampling intervals, transients, or another operating mode can produce small readings. The rule must be supported by training data for the relevant device and sampling interval.

### 8.4 Example: classification and power should agree

Let $s_i(t) \in \{0, 1\}$ denote a candidate operating state: 0 means off or standby, and 1 means active. These states are inferred at test time, not taken from held-out labels. For a simplified two-state appliance, one possible consistency rule is

$$\begin{aligned} s_i(t) P_{i,\text{on,min}} \le p_i(t) \le (1 - s_i(t)) P_{i,\text{off,max}} \\ + s_i(t) P_{i,\text{on,max}}. \end{aligned} \tag{53}$$

**Meaning.** If the candidate state is off, power must lie in the off/standby range. If it is active, power must lie in the allowed active range. The bounds are selected from training data and device knowledge.

**Example.** Suppose the illustrative off maximum is 10 W and the active range is 1000–1800 W. State $s = 0$ with predicted power 1500 W violates the rule. State $s = 1$ with power 1500 W satisfies it. A 100 W reading fits neither state under this particular toy model; this could indicate a false pulse, or it could show that the two-state model is too simple. These bounds must not be imposed universally on all microwaves or on multi-stage devices.

For real equipment, use appropriate states and account for interval averaging. A model may predict states and powers jointly from a shared representation. The constraint should connect those outputs; independently training two heads does not ensure agreement. Binary states and event extraction are discrete operations. Strict enforcement may need discrete inference or a suitable relaxation rather than ordinary input-gradient descent.

### 8.5 Example: a duration penalty for short false events

Let $\mathcal{E}_i$ be the set of complete candidate active events for appliance $i$. Let $d_e$ be an event duration in seconds and $d_{i,\min} > 0$ a chosen minimum duration for the illustrative model. A simple soft penalty is

$$R_{\mathrm{dur},i} = \sum_{e \in \mathcal{E}_i} \left[ \max\left(0, \frac{d_{i,\min} - d_e}{d_{i,\min}}\right) \right]^2 . \tag{54}$$

**Meaning.** A complete event shorter than the chosen minimum incurs a positive penalty. Longer events receive zero penalty from this term; that alone does not make them correct.

**Example.** With a hypothetical minimum of 30 s, an isolated 6 s event receives $((30 - 6)/30)^2 = 0.64$. A 60 s event receives zero. These values explain the formula only. The minimum must be fitted and validated. An event cut by a window boundary is not a complete short event and needs separate treatment.

Equation (54) is a proposed teaching penalty, not an equation taken from Wong et al. [12]. Their paper models duration-dependent state transitions. Our simple penalty also does not replace a full model of off durations, variable modes, or operating cycles. Because event extraction is discrete, it can be used as a diagnostic or within a suitable constrained solver; a differentiable training loss requires further design.

### 8.6 How these ideas enter training and inversion

The objective in Equation (29) can include state and duration information in its device penalties. A practical research procedure is:

1. **Audit the training labels.** Inspect real events, off periods, power ranges, state definitions, and sampling intervals for all five appliances. Estimate the training residual using Equation (28).

2. **Learn device trajectories.** Train a temporal prior on each device's varied windows. One option is an encoder–decoder that reconstructs clean appliance windows and constrains its codes. Reconstruction training alone does not guarantee that every possible code generates a valid event.

3. **Choose the constraints.** Specify which conditions are hard bounds and which are soft preferences. Define state labels from training channels using the declared convention. Fit background assumptions using training data as well.

4. **Freeze the learned priors.** For a new aggregate, initialise candidate codes, optionally from the existing direct model. Held-out target powers and derived true residuals are unavailable to inference.

5. **Refine the explanation.** Update the candidate inputs so that their generated trajectories fit the aggregate and obey the selected device/background conditions. Use an appropriate solver for the chosen smooth or discrete variables.

6. **Evaluate the attribution.** Check power error, false and missed events, state consistency, timing, background allocation, and computation cost against held-out data.

CNNs can implement the temporal generators, but the useful ingredients are learned device behaviour and validated constraints. A generator that emits a convincing event at the wrong time or for the wrong device has still made a NILM error.

### 8.7 Related literature and the limits of a novelty claim

Bao et al. [11] directly discuss unrealistic NILM output shapes and propose pretrained appliance sequence generators using a generative adversarial network (GAN). Their work is closely related to the present motivation. It reports preliminary results without strong evidence of consistent improvement over the examined direct baseline. It does not establish the proposed five-target joint inverse optimisation. Their training difficulties also motivate including both inactive and active windows.

Kelly and Knottenbelt [10] show that direct learning can use temporal structure and activation-level outputs. Kolter and Jaakkola [6] incorporate additive device states and unmodelled observations. Zhong et al. [7] add per-source aggregate constraints. Wong et al. [12] model state duration. Ren et al. [1] provide the frozen-model input-optimisation mechanism. These are complementary foundations; none alone proves the proposed project works or is new.

The contribution therefore needs a specific method and a demonstrated benefit. A useful target is fewer false pulses and state conflicts, with correct event attribution and an acceptable power-error/runtime trade-off. Merely adding a sum loss, using a CNN, generating plausible shapes, or adding a residual is insufficient to establish novelty.

## 9 Formula-by-formula reading guide

This section is a plain-language guide to the main formulae. The scalar NA calculations and the state/duration examples are explained directly where they appear. In an optimisation formula, read arg min as "find the value that makes the quantity after it as small as possible." A hat, such as $\hat{\mathbf{p}}$, means an estimate; a tilde, such as $\tilde{\mathbf{p}}$, means an initial or intermediate estimate. Superscript $(m)$ identifies a training example and superscript $(k)$ identifies an optimisation iteration.

### 9.1 Basic forward and inverse notation

**Equation** (1)**:** $\mathbf{y} = f(\mathbf{x})$**. Meaning.** A forward model maps a cause $\mathbf{x}$ to an observable effect $\mathbf{y}$. In NILM, $\mathbf{x}$ is a collection of appliance power trajectories and $\mathbf{y}$ is the whole-house aggregate trajectory.

**Example.** If the refrigerator is 100 W, microwave is 1500 W, and all other relevant power is 400 W, the forward rule returns $y = 100 + 1500 + 400 = 2000$ W. This says what the meter should read if the hidden appliance powers are known.

**Equation** (2)**:** $\hat{\mathbf{x}} \in \arg\min_{\mathbf{x}} \mathcal{L}(f(\mathbf{x}), \mathbf{y}_{\text{obs}}) + R(\mathbf{x})$**. Known.** The observed aggregate $\mathbf{y}_{\text{obs}}$ and the forward rule $f$.

**Unknown and optimised.** The candidate hidden input $\mathbf{x}$; in NILM, this means candidate appliance trajectories.

**Meaning.** The first term asks, "does this candidate produce the observed aggregate?" The second term asks, "is this candidate a reasonable appliance explanation?" The symbol $\in$ is used because multiple different candidates may achieve the same smallest objective.

**Example.** For an observed one-sample aggregate of 2000 W, take a squared mismatch $\mathcal{L} = (f(\mathbf{x}) - 2000)^2$. A candidate with refrigerator = 100 W, microwave = 1500 W, and residual = 400 W has mismatch 0. A candidate with refrigerator = 100 W, microwave = 1900 W, and residual = 0 W also has mismatch 0. A prior $R(\mathbf{x})$ is needed to prefer the candidate more compatible with learned appliance and background behaviour.

**Equations** (5) **and** (6)**: physical power balance. Meaning.** Equation (5) gives the balance at one time sample, while Equation (6) writes the same relationship for a whole window. The

matrix $A$ adds the five target-appliance trajectories sample by sample. The residual $\mathbf{r}$ combines unrecorded appliance power and discrepancies.

**Example.** At time $t = 3$, $y_{\text{agg}}(3) = 2000$ W can equal $0 + 100 + 0 + 0 + 1500 + 400$ W. Across the six-sample example window, $A\mathbf{x}$ adds the six values of each target trajectory, and then $\mathbf{r}$ is added elementwise.

### 9.2 NA equations

**Equation** (14)**: the NA update. Known and frozen.** The trained forward-model parameters $\theta^*$ and the observed target $\mathbf{y}_{\text{obs}}$.

**Updated.** The candidate input $\hat{\mathbf{x}}^{(k)}$, not the neural-network weights.

**Meaning.** The gradient $\nabla_{\mathbf{x}}\mathcal{L}$ tells how a small change in each input component changes the loss. Subtracting the gradient, scaled by $\alpha_k$, moves locally downhill. This update is an optimisation procedure at test time; it is not a second neural network.

**Example.** Suppose the current predicted aggregate is 1300 W but the observation is 2000 W. The mismatch is large. In a meaningful learned forward model, the gradient could indicate that increasing a microwave-related input is a more effective way to raise the predicted output than changing another input. For the exact sum alone, however, every appliance receives the same aggregate gradient; this limitation is explained by Equation (23).

**Equation** (19)**: boundary loss. Meaning.** This term is zero while a candidate input remains within an allowed training-data range and positive outside that range. It discourages the optimiser from exploiting regions where a learned surrogate may be inaccurate.

**Example.** If microwave training windows show power between 0 and 1800 W, a candidate microwave value of 2500 W receives a positive boundary penalty. This does not prove that every value under 1800 W is realistic; it only excludes a basic out-of-range solution.

### 9.3 Why the sum-only model is ambiguous

**Equation** (22)**: learned summation. Meaning.** A neural network given five appliance powers and a residual can learn to add them. This is easy, but it contributes little because Equation (5) already gives the exact relation.

**Example.** Feeding $[0, 100, 0, 0, 1500]$ W and residual 400 W into the network should produce 2000 W. A calculator, not a CNN, can do this exactly. Therefore low error for this forward-training task does not demonstrate useful NILM disaggregation ability.

**Equation** (23)**: equal gradients of a sum. Meaning.** Changing any one appliance by 1 W changes the aggregate by 1 W. The aggregate constraint alone therefore supplies no label saying which appliance caused a missing or excess watt.

**Example.** If the predicted sum is 700 W below the aggregate, increasing the microwave by 1 W, the refrigerator by 1 W, or the residual by 1 W all reduce the aggregate mismatch by the same first-order amount. Appliance-specific information must come from priors, additional measurements, or a direct prediction model.

**Equation** (24)**: unchanged aggregate directions. Meaning.** Any redistribution $\Delta\mathbf{x}$ between appliances and $\Delta\mathbf{r}$ in the residual satisfying this equation leaves the observed aggregate unchanged. These are directions in which the inverse problem is not identifiable from aggregate power alone.

**Example.** At one time sample, increase microwave power by 500 W and decrease residual by 500 W. The aggregate does not change. Both allocations cannot be distinguished using the sum constraint alone.

### 9.4 Direct NILM and learned shape priors

**Equation** (25)**: direct supervised NILM prediction. Known.** The aggregate window $\mathbf{y}_{\text{agg}}$.

**Output.** Five predicted appliance trajectories, one per target appliance.

**Meaning.** A direct model uses correlations learned from labelled examples to predict appliance powers in one forward pass. It does not explicitly solve an optimisation problem at test time.

**Example.** A direct CNN may see the pattern $[400, 400, 2000, 2000, 400, 400]$ W and predict a refrigerator baseline plus a two-sample microwave plateau. This is where CNN temporal feature learning is valuable.

**Equation** (27)**:** $\mathbf{p}_i = G_i(\mathbf{z}_i)$**. Known and frozen at test time.** The trained generator $G_i$.

**Updated.** Its latent input $\mathbf{z}_i$, not each of the $T$ power samples independently.

**Meaning.** The generator maps a code to a complete appliance trajectory. A compact code is one way to restrict the search, but its dimension is a design choice. The aim is to favour patterns resembling the appliance class.

**Example.** Instead of freely selecting six microwave powers $[p(1), \ldots, p(6)]$, a code can generate a coherent pattern close to $[0, 0, 1500, 1500, 0, 0]$. If the decoder has been properly trained, it should be harder to generate a refrigerator-like 100 W baseline as a microwave trajectory.

**Equation** (28)**: construction of training residuals. Known during training.** The observed aggregate and all five target-appliance labels.

**Meaning.** Subtract the labelled targets from the aggregate to obtain a combined residual training trajectory. This is allowed during training, but it is not available at test time because the target labels are then unknown.

**Example.** At sample 3 in the worked example, $r = 2000 - (0 + 100 + 0 + 0 + 1500) = 400$ W. This value is not a label for one particular other appliance; it is the total of all unrecorded contributions and discrepancies.

**Equation** (29)**: constrained latent-space inverse objective. Known.** The observed aggregate and the frozen appliance/background generators.

**Unknown and optimised.** The latent variables $\mathbf{z}_1, \ldots, \mathbf{z}_5, \mathbf{z}_{\text{bg}}$.

**Meaning.** The first line says that generated appliance and background trajectories should add to the observed aggregate. The second line says that the latent codes should remain plausible under the learned appliance and background priors. This is the proposed source-separation formulation.

**Example.** A candidate code that generates a 1500 W microwave plateau at samples 3–4 plus a 400 W background may fit both terms well. A code that makes the background generator explain all 2000 W may match the aggregate but should be penalised if that residual pattern is atypical. Whether the penalty truly prevents collapse must be tested, not assumed.

### 9.5 Formulae in the numerical example

**Equation** (30)**: observed aggregate window. Meaning.** This is the only six-sample power vector provided to the inference algorithm in the worked example.

**Example.** The two central 2000 W samples suggest that an additional high-power load may be active, but the aggregate alone does not identify the load as a microwave.

**Equation** (31)**: example power balance. Meaning.** For this particular illustration, only refrigerator, microwave, and residual are nonzero. It is a simplified instance of Equation (6).

**Example.** At samples 1 and 2, $400 = 100 + 0 + 300$ W. At samples 3 and 4, $2000 = 100 + 1500 + 400$ W.

**Equation** (33)**: direct-model training loss. Known during training.** $M$ labelled aggregate windows and their five appliance trajectories.

**Updated.** The direct-model parameters $\phi$.

**Meaning.** For every training window and appliance, compare the predicted trajectory with its labelled trajectory, square the differences, and add them. Minimising the result encourages accurate appliance predictions.

**Example.** If a true microwave value is 1500 W but the model predicts 1000 W at one sample, that sample contributes $(1000 - 1500)^2 = 250{,}000$ W$^2$ before averaging. Unlike aggregate-only training, the loss explicitly tells the model that the microwave was wrong.

**Equations** (36) **and** (38)**: direct prediction and implied residual. Meaning.** These equations show an imperfect direct estimate and then calculate the portion of aggregate not explained by it. The residual is not observed directly; it is inferred by subtraction after a candidate target allocation is chosen.

**Example.** At sample 3, the direct estimate allocates 100 W to refrigerator and 1000 W to microwave. The unexplained amount is $2000 - 100 - 1000 = 900$ W. The hidden truth has 400 W residual, so the direct model has assigned 500 W too little to the microwave.

**Equation** (39)**: sum-only objective. Meaning.** This objective only checks whether candidate sources reproduce the aggregate. It contains no term that recognises microwave, refrigerator, or background shapes.

**Example.** The hidden-truth allocation, the 1900 W microwave allocation, and the all-background allocation all make this objective equal to zero at samples 3–4. Thus, a perfect value of this objective is not proof of successful disaggregation.

**Equation** (41)**: sample-level gradient example. Meaning.** The reconstruction gradient is $2e$ for every source at one sample. Its sign tells all sources to increase when the total is too small and to decrease when the total is too large, but it cannot assign responsibility.

**Example.** With the candidate refrigerator = 100 W, microwave = 1000 W, residual = 200 W, the error is $e = -700$ W. Each derivative is $2e = -1400$. The numerical value is the same for kettle, refrigerator, dishwasher, washing machine, microwave, and residual.

**Equations** (44)**,** (45)**, and** (46)**: generated shapes. Meaning.** These equations separate the generator architecture from its latent code, then show toy examples of class-specific output shapes. They demonstrate why a generator can reduce the search space from arbitrary pointwise powers to structured trajectories.

**Example.** A microwave code may generate a high-power two-sample plateau; a refrigerator code may generate a low, sustained baseline. The bounds $0 \le a \le 1800$ and $0 \le b \le 300$ W are pedagogical values, not values to impose without auditing the actual data.

**Equation** (47)**: prior-constrained objective. Meaning.** This is Equation (29) written again for the numerical example. The reconstruction term alone checks the meter reading. The two $\Omega$ terms express the preference for ordinary appliance and background latent codes.

**Example.** If an all-background solution requires an unusual background latent code while the microwave solution requires a typical microwave-event code, the prior terms can favour the

microwave explanation even if both reconstruct 2000 W. If the generators overlap strongly, the objective may still be ambiguous.

**Equation** (48)**: latent-variable update. Known and frozen.** Generator weights and observed aggregate.

**Updated.** The stacked latent vector $\mathbf{z}$.

**Meaning.** This is gradient descent in latent space. The chain rule passes the loss gradient through each frozen generator to its latent input. A compact code can reduce the number of unknowns, but the problem can still be non-convex and sensitive to initialisation.

**Example.** Beginning from the direct-CNN output can initialise the microwave latent code near a two-sample event. Optimisation can then increase its generated amplitude towards 1500 W while adjusting the background latent code towards 400 W, if the prior and gradients support that allocation.

**Equation** (51)**: desired result. Meaning.** These are the target estimates that would match the illustrative hidden trajectories. The equation is a desired outcome, not a guarantee supplied by the optimisation formulation.

**Example.** Adding the three displayed vectors gives $[400, 400, 2000, 2000, 400, 400]^\top$ W exactly. The other three target trajectories remain zero.

**Equation** (52)**: microwave MAE. Known only for evaluation.** The held-out microwave ground-truth trajectory.

**Meaning.** Take the absolute power error at every sample and average it. This measures whether the model assigned power correctly to the microwave, independently of whether the total aggregate was reconstructed.

**Example.** Predicting 1900 W instead of 1500 W at samples 3 and 4 yields errors $0, 0, 400, 400, 0, 0$ W. Their average is $800/6 \approx 133.3$ W, even though aggregate reconstruction can be perfect.

## 10 Research risks and experimental requirements

| Risk | Cause | Required test |
|---|---|---|
| Learned sum has no value | Addition is known exactly | Compare against exact summation under identical priors and optimisation budgets |
| Background collapse | A flexible background can explain all power | Report the fraction of energy assigned to background and target-wise errors |
| Target inflation | A restricted residual forces background into targets | Evaluate separately during strong non-target events |
| Unrealistic trajectories | Reconstruction alone permits arbitrary shapes | Inspect waveform plausibility and use appliance-state/event metrics |
| Poor cross-house generalisation | Background appliances and usage differ by household | Use household-disjoint validation and test splits |
| Slow or unstable inference | Non-convex latent optimisation depends on initialisation | Report runtime, restarts, and variability across initialisations |

The recommended comparisons are:

1. a direct supervised baseline, such as the existing `MARTNILM` configuration;

2. the same direct predictions with simple thresholding and short-event filtering, using validation-selected settings;
3. direct prediction followed by constrained refinement using exact power balance;
4. a prior-free optimisation baseline, to expose the ambiguity of summation alone;
5. a generative-prior inverse model with the same data split and target definition;
6. ablations with no background model, a weak background model, and a learned background prior.

### 10.1 A staged first experiment

First record the direct baseline's actual failure cases: false short pulses, missed events, incorrect durations, and state–power disagreements. Next compare simple filtering with explicit constrained refinement. This tests whether the proposed rules provide value beyond an inexpensive cleanup. It is a low-cost pilot of the constraints, not yet the complete deep inverse model. If useful improvements remain, train the generators and compare their joint inverse inference under the same evaluation protocol.

Keep model training, prior fitting, and threshold selection separate from held-out houses. If a duration rule removes false events but also deletes true short events, that trade-off must be reported. If filtering matches the full method at much lower cost, the extra model complexity needs another justification.

### 10.2 Metrics that match the research goal

| Question | Suggested measure | What it adds |
|---|---|---|
| Are powers correct? | Per-device MAE and energy error. | Measures amplitude and consumption errors. |
| Are active states correct? | State precision, recall, and F1 score. | Separates false activations from missed activity. |
| Are events correct? | Event precision/recall, onset/offset error, duration error, and event count. | Tests the timing and fragmentation hidden by one average. |
| Are outputs internally consistent? | State–power conflicts and rule violations. | Tests declared logic; these checks do not prove correct device attribution. |
| Is the meter explained? | Aggregate mismatch and background allocation. | Detects unexplained power and background collapse. |
| Is inference practical? | Runtime and variability over initialisations. | Shows the cost and stability of inverse optimisation. |

Declare sampling intervals, state thresholds, event-matching tolerances, and boundary handling before the final test. Waveform plots should include representative successes and failures, not only smooth-looking outputs. A successful method should reduce errors in actual device attribution or offer a documented trade-off such as fewer false events at similar MAE. Lower aggregate error or fewer rule violations alone is insufficient.

## 11 Conclusion and recommended project statement

The original idea—learn appliance powers to aggregate power and reverse the process—contains a valid inverse-problem intuition but is insufficient if the learned forward model only reproduces addition. In power-only NILM, the forward law is simple while the inverse is fundamentally non-unique. Therefore, a learned sum does not make the inverse identifiable.

The supervisor's direction remains a reasonable research hypothesis when the learned forward component describes appliance operation and waveform generation. A forward process is physically natural, and inverse inference from its measurements is also legitimate. The project should test whether device-shape, duration, and state constraints improve real appliance recovery, especially with unrecorded background loads or unseen houses. A concise project statement is:

> *Can learned appliance-behaviour priors and constrained inverse optimisation reduce false events and state–power conflicts while retaining or improving five-appliance NILM accuracy in houses with unrecorded loads?*

The goal includes correct waveform shape, event identity, and aggregate consistency. A realistic-looking waveform assigned to the wrong appliance remains an error. The comparisons above should establish whether the benefits justify the added training and inference cost. Existing work supports the ingredients; the specific contribution and its novelty still require a focused literature comparison and experiments.

## References


[1] S. Ren, W. J. Padilla, and J. Malof. Benchmarking deep inverse models over time, and the Neural-Adjoint method. In *Advances in Neural Information Processing Systems*, 2020. https://proceedings.neurips.cc/paper_files/paper/2020/file/007ff380ee5ac49ffc34442f5c2a2b86-Paper.pdf

[2] D. Liu, Y. Tan, E. Khoram, and Z. Yu. Training deep neural networks for the inverse design of nanophotonic structures. *ACS Photonics*, 5(4):1365–1369, 2018. https://doi.org/10.1021/acsphotonics.7b01377

[3] A. Bora, A. Jalal, E. Price, and A. G. Dimakis. Compressed sensing using generative models. In *Proceedings of the 34th International Conference on Machine Learning*, pages 537–546, 2017. https://proceedings.mlr.press/v70/bora17a.html

[4] V. Narayanaswamy, J. J. Thiagarajan, R. Anirudh, and A. Spanias. Unsupervised audio source separation using generative priors. In *Interspeech*, pages 2657–2661, 2020. https://doi.org/10.21437/Interspeech.2020-3115

[5] V. Jayaram and J. Thickstun. Source separation with deep generative priors. In *Proceedings of the 37th International Conference on Machine Learning*, pages 4724–4735, 2020. https://proceedings.mlr.press/v119/jayaram20a.html

[6] J. Z. Kolter and T. Jaakkola. Approximate inference in additive factorial HMMs with application to energy disaggregation. In *Proceedings of the 15th International Conference on Artificial Intelligence and Statistics*, pages 1472–1482, 2012. https://proceedings.mlr.press/v22/zico12.html

[7] M. Zhong, N. Goddard, and C. Sutton. Signal aggregate constraints in additive factorial HMMs, with application to energy disaggregation. In *Advances in Neural Information Processing Systems*, 2014. https://proceedings.neurips.cc/paper_files/paper/2014/hash/bf64575b42486eb72efdfa0091be60a5-Abstract.html

[8] J. Adler and O. Öktem. Learned primal-dual reconstruction. *IEEE Transactions on Medical Imaging*, 37(6):1322–1332, 2018. https://doi.org/10.1109/TMI.2018.2799231

[9] L. Ardizzone, J. Kruse, C. Rother, and U. Köthe. Analyzing inverse problems with invertible neural networks. In *International Conference on Learning Representations*, 2019. https://arxiv.org/abs/1808.04730

[10] J. Kelly and W. Knottenbelt. Neural NILM: Deep neural networks applied to energy disaggregation. In *Proceedings of the 2nd ACM International Conference on Embedded Systems for Energy-Efficient Built Environments*, pages 55–64, 2015. https://doi.org/10.1145/2821650.2821672

[11] K. Bao, K. Ibrahimov, M. Wagner, and H. Schmeck. Enhancing neural non-intrusive load monitoring with generative adversarial networks. *Energy Informatics*, 1(Suppl 1), article 18, 2018. https://doi.org/10.1186/s42162-018-0038-y

[12] Y. F. Wong, T. Drummond, and Y. A. Şekercioğlu. Real-time load disaggregation algorithm using particle-based distribution truncation with state occupancy model. *Electronics Letters*, 50(9):697–699, 2014. https://doi.org/10.1049/el.2013.3967

[13] D. P. Kingma and M. Welling. Auto-encoding variational Bayes. *arXiv preprint arXiv:1312.6114*, 2013. https://arxiv.org/abs/1312.6114